ARQUITECTURA v1.0

Arquitectura de Agentes

Diagramas de Flujo

Intrale Platform — Sistema de Agentes Autónomos Orquestados

Proyecto	Fecha	Agentes	Stack
intrale/platform	2026-03-01	22 skills + 10 hooks	Claude Code + Node.js + PowerShell

Indice

1. Resumen Ejecutivo
2. Inventario de Agentes
3. Diagrama de Flujo: Ciclo de Sprint
4. Diagrama de Flujo: Implementacion de un Issue
5. Diagrama de Interaccion entre Agentes
6. Sistema de Hooks
7. Integracion Telegram
8. Orquestacion y Worktrees
9. Flujo de Datos Persistentes
10. Modelos y Costos
11. Protecciones y Gates

1. Resumen Ejecutivo

Intrale Platform implementa un **sistema de agentes IA autonomos orquestados** que automatiza el ciclo completo de desarrollo de software: planificacion, implementacion, testing, QA, code review y delivery. Los agentes operan en **worktrees Git aislados**, se comunican via **Telegram** y son monitoreados en tiempo real.

22	10	3	5
SKILLS / AGENTES	HOOKS AUTOMATICOS	AGENTES EN PARALELO	SCRIPTS ORQUESTACION

Ciclo autonomo: El sistema puede ejecutar sprints completos sin intervencion humana. El usuario monitorea desde Telegram y solo interviene para aprobar permisos criticos o decidir si iniciar un nuevo sprint.

2. Inventario Completo de Agentes

2.1 Clasificacion por Tier

Tier 1 — Investigacion y Planificacion

Skill	Nombre	Modelo	Funcion Principal	Modos
`/guru`	Guru	SONNET	Investigacion tecnica con Context7 + codebase	investigar
`/planner`	Planner	SONNET	Planificacion estrategica, sprints, propuestas	sprint, planificar, proponer
`/doc`	Doc	SONNET	Gestion unificada de backlog	nueva, refinar, triaje, estado
`/historia`	Historia	SONNET	Crear historias de usuario como issues GitHub	crear
`/refinar`	Refinar	SONNET	Refinar issues existentes con estructura estandar	refinar
`/priorizar`	Priorizar	SONNET	Triaje masivo de issues sin labels	triaje

Tier 2 — Desarrollo por Plataforma

Skill	Nombre	Modelo	Funcion Principal	Modulo
`/branch`	Branch	HAIKU	Guardian Git, crea ramas, protege main	Todos
`/backend-dev`	BackendDev	SONNET	Backend Ktor, DynamoDB, Cognito, Lambda	:backend, :users
`/android-dev`	AndroidDev	SONNET	Android Compose, Material3, flavors	:app (androidMain)
`/ios-dev`	iOSDev	SONNET	iOS ComposeUIViewController, framework	:app (iosMain)
`/web-dev`	WebDev	SONNET	Web Kotlin/Wasm, PWA, Webpack	:app (wasmJsMain)
`/desktop-dev`	DesktopDev	SONNET	Desktop JVM, Window API, Swing	:app (jvmMain)
`/builder`	Builder	HAIKU	Build, compilacion, verificaciones	Todos

Tier 3 — Testing y Calidad

Skill	Nombre	Modelo	Funcion Principal	Gate
`/tester`	Tester	HAIKU	Tests unitarios, cobertura Kover	NO
`/qa`	QA E2E	SONNET	Tests E2E contra entorno real + video	GATE
`/review`	Review	SONNET	Code review automatizado de PRs	NO
`/po`	Product Owner	SONNET	Validacion de negocio, criterios BDD	NO
`/ux`	UX Specialist	SONNET	Auditoria UX, benchmarking, tendencias	NO

Tier 4 — Entrega y Cierre

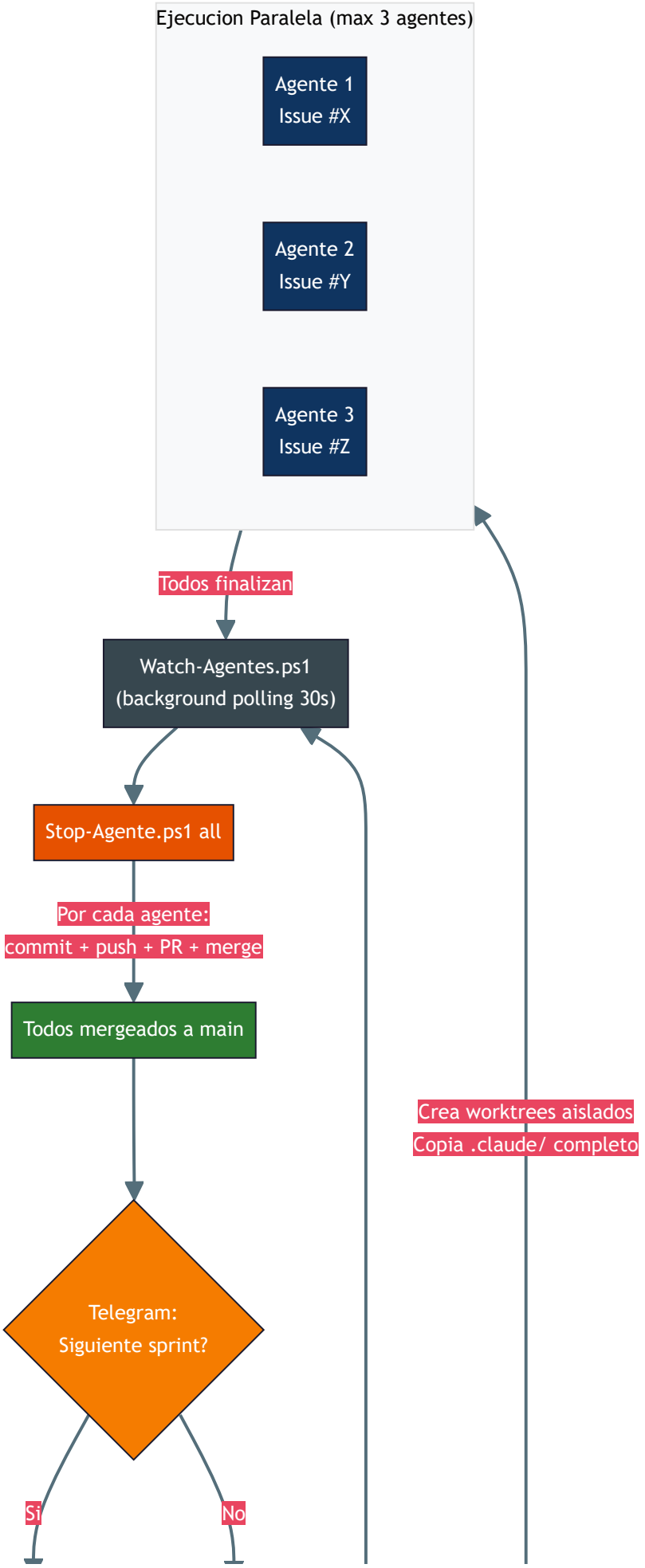
Skill	Nombre	Modelo	Funcion Principal	Gate QA
`/delivery`	DeliveryManager	HAIKU	Commit + push + PR + merge en un paso	OBLIGATORIO

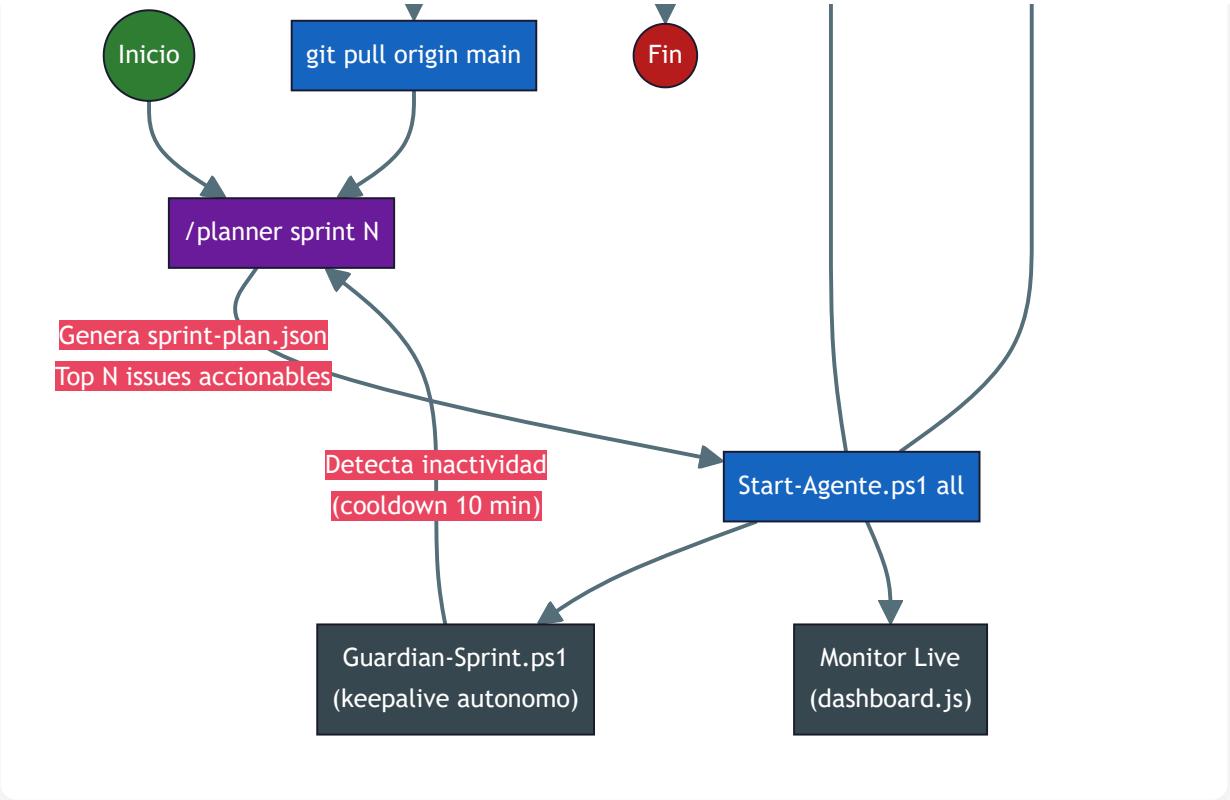
Tier 5 — Monitoreo, Seguridad y Mantenimiento

Skill	Nombre	Modelo	Funcion Principal
`/monitor`	Monitor	HAIKU	Dashboard multi-sesion en tiempo real
`/auth`	Auth	HAIKU	Gestion de permisos de Claude Code
`/cleanup`	Cleanup	HAIKU	Limpieza segura de workspace

3. Diagrama de Flujo: Ciclo de Sprint Autonomo

El siguiente diagrama muestra el ciclo completo de un sprint, desde la planificacion hasta el cierre y la decision de iniciar el siguiente sprint.


Ejecucion Paralela (max 3 agentes)
Agente 1
Issue #X
Agente 2
Issue #Y
Agente 3
Issue #Z
Todos finalizan
Watch-Agentes.ps1
(background polling 30s)
Stop-Agente.ps1 all
Por cada agente:
commit + push + PR + merge
Todos mergeados a main
Telegram:
Siguiente sprint?
Si
No
Crea worktrees aislados
Copia .claude/ completo

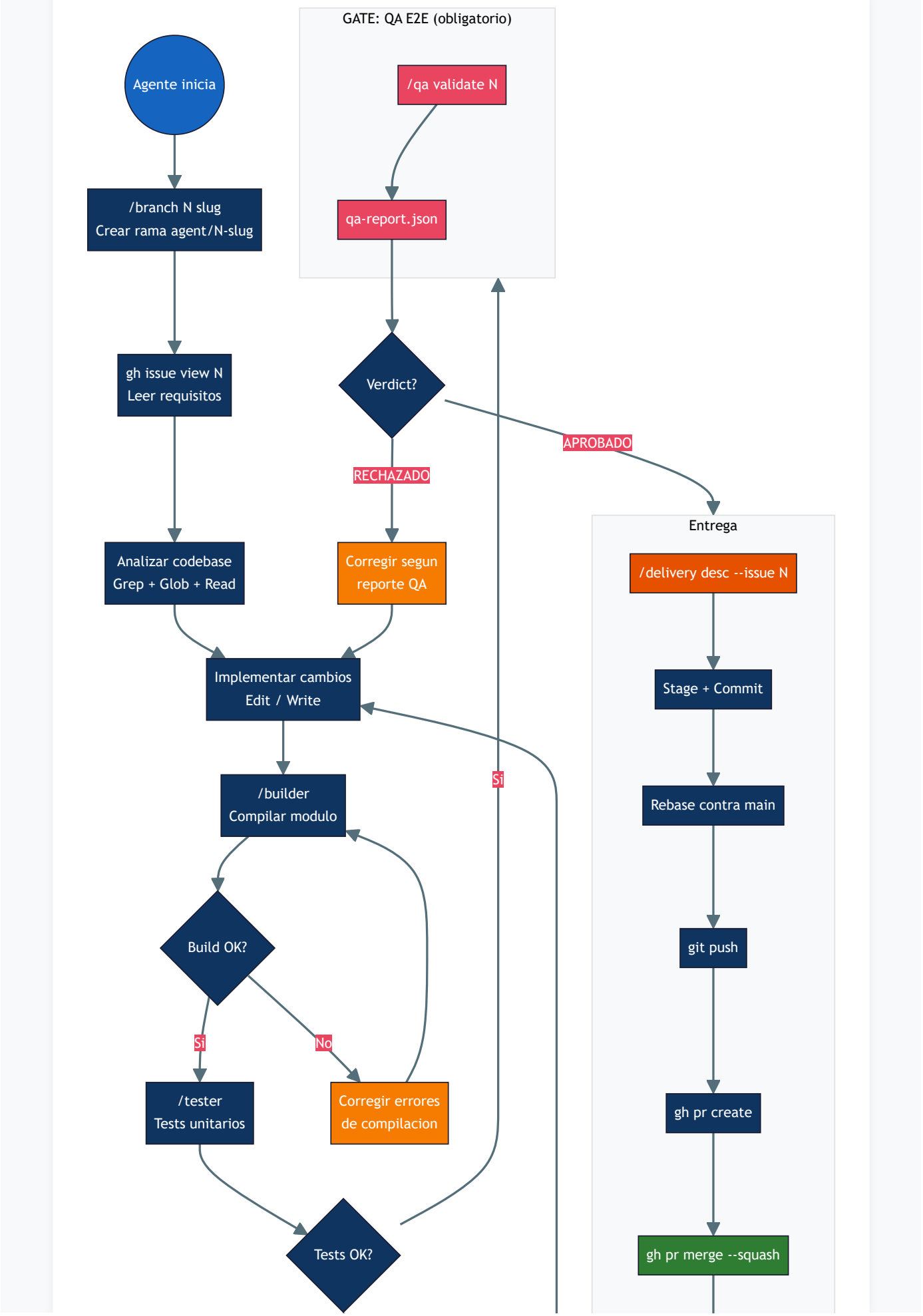


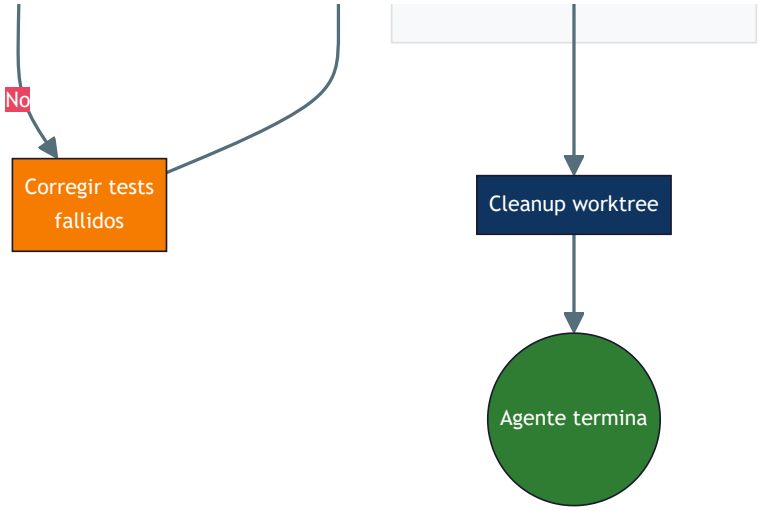
Planificacion · Orquestacion · Agentes de trabajo · Monitoreo/Background · Delivery · Exito

Ciclo continuo: Cuando todos los agentes terminan, Watch-Agentes pregunta al usuario via Telegram si desea iniciar otro sprint. Si el usuario no responde y hay inactividad, Guardian-Sprint relanza automaticamente el planner (con cooldown de 10 minutos).

4. Diagrama de Flujo: Implementacion de un Issue

Cada agente individual sigue este flujo al implementar un issue. Las flechas rojas indican gates (puntos de bloqueo obligatorios).

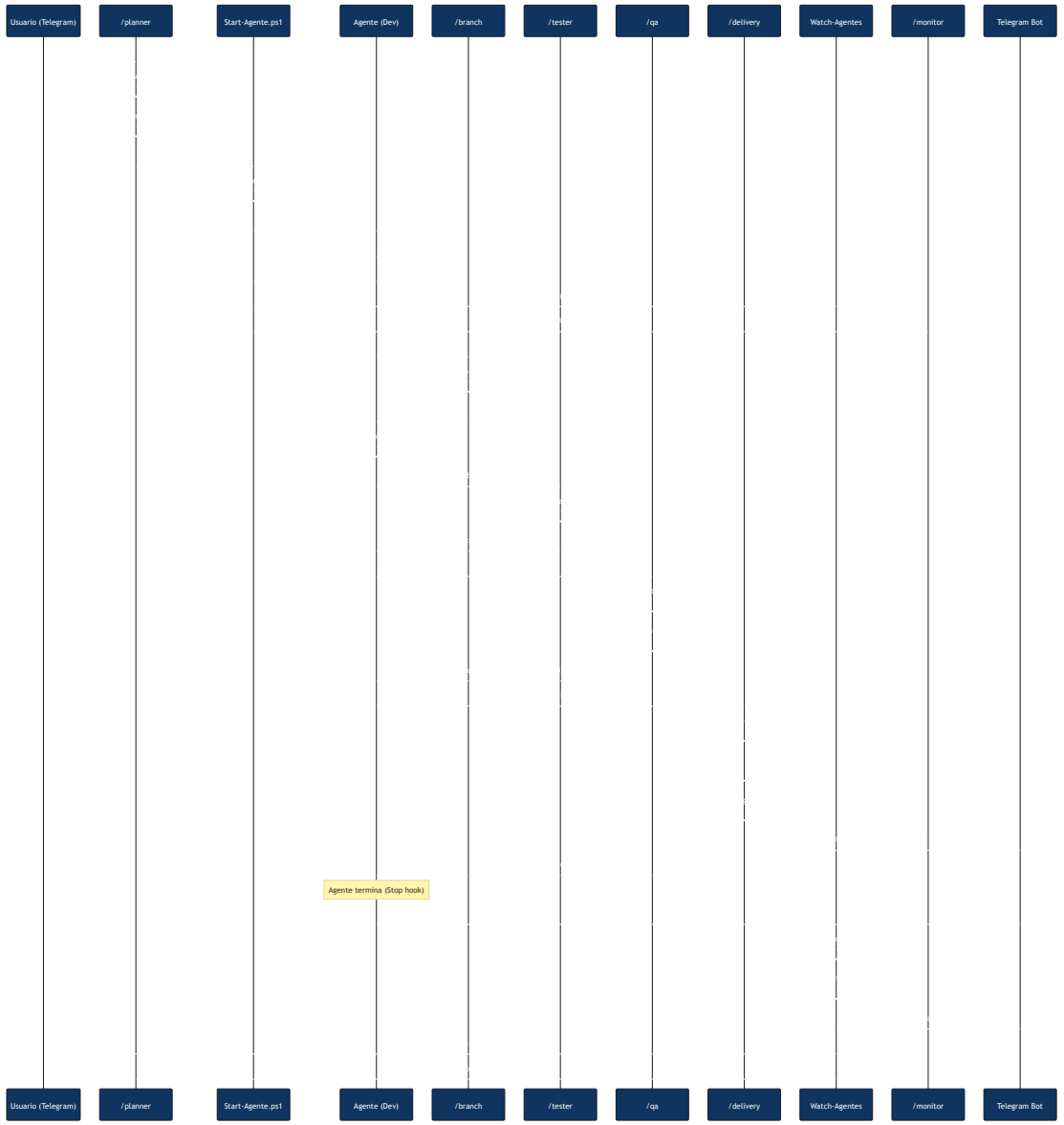

GATE: QA E2E (obligatorio)
Agente inicia
/qa validate N
/branch N slug
Crear rama agent/N-slug
qa-report.json
gh issue view N
Leer requisitos
Verdict?
APROBADO
RECHAZADO
Entrega
Analizar codebase
Grep + Glob + Read
Corregir segun
reporte QA
/delivery desc --issue N
Implementar cambios
Edit / Write
Stage + Commit
Si
/builder
Compilar modulo
Rebase contra main
Build OK?
git push
Si
No
/tester
Tests unitarios
Corregir errores
de compilacion
gh pr create
Tests OK?
gh pr merge --squash



Gate critico: `/delivery` BLOQUEA el merge si `qa/evidence/<issue>/qa-report.json` no existe o su veredicto es "RECHAZADO". El agente debe corregir y re-ejecutar `/qa validate` hasta obtener aprobacion.

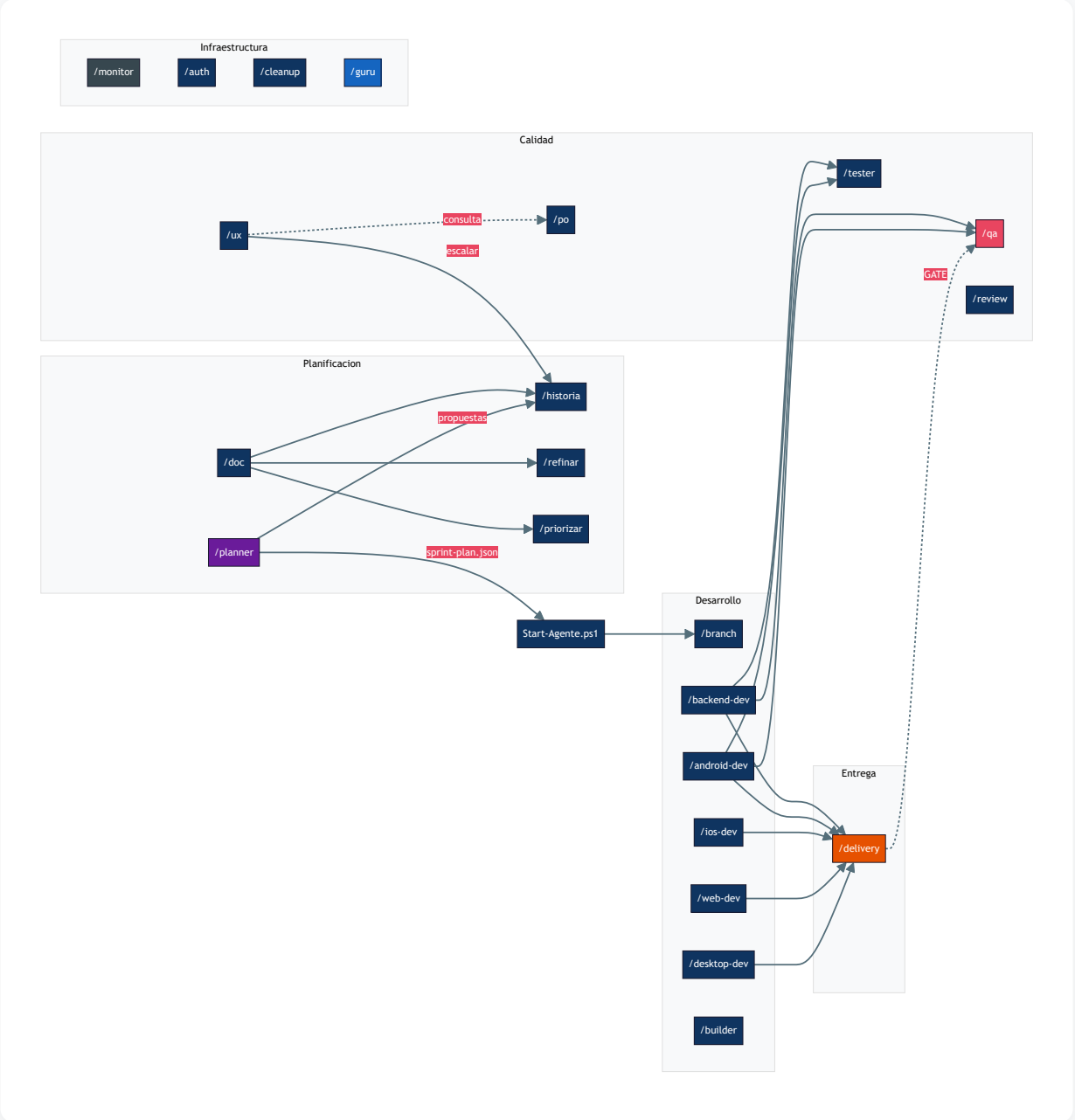
5. Diagrama de Interaccion entre Agentes

Este diagrama de secuencia muestra como interactuan los distintos agentes durante un sprint tipico, incluyendo las comunicaciones con Telegram y los hooks automaticos.



5.1 Mapa de Invocaciones Cruzadas

Quien invoca a quien en el ecosistema de agentes:



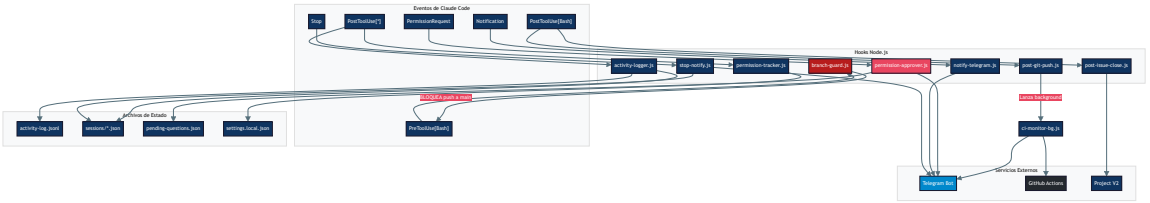
5.2 Tabla de Dependencias

Skill Origen	Invoca	Tipo	Cuando
`/doc`	`/historia`, `/refinar`, `/priorizar`	Delegacion	Segun modo (nueva, refinar, triaje)
`/planner sprint`	`Start-Agente.ps1`	Orquestacion	Al generar sprint-plan.json
`/planner proponer`	`/historia` (via Telegram)	Indirecto	Usuario aprueba propuesta
`/ux escalar`	`/historia`	Creacion	Detecta problemas UX graves
Agentes Dev	`/branch`	Prerequisito	Siempre (primer paso)

Agentes Dev	`/tester`	Verificacion	Post-implementacion
Agentes Dev	`/qa validate`	Gate	Pre-delivery
Agentes Dev	`/delivery`	Cierre	Al finalizar
`/delivery`	`/qa` (lectura)	GATE	Verifica qa-report.json
`/ux`	`/po`	Consulta	Para reglas de negocio

6. Sistema de Hooks

Los hooks son scripts Node.js que se disparan automaticamente en eventos especificos de Claude Code. Constituyen la capa de automatizacion invisible que conecta todas las piezas.



6.1 Detalle de Cada Hook

branch-guard.js

PreToolUse[Bash]

Funcion: Intercepta comandos `git push`. Si la rama actual es `main` o `master`, BLOQUEA la ejecucion y devuelve error. Solo permite push a ramas `agent/*`, `codex/*`, `feature/*`, etc.

permission-approver.js

PermissionRequest

Funcion: Cuando Claude necesita aprobacion para una herramienta, envia mensaje a Telegram con botones inline (Permitir / Rechazar). Implementa retry con escalada de urgencia: 4 intentos en ~65 minutos. Si no hay respuesta, cae a prompt local en consola.

post-git-push.js

PostToolUse[Bash]

Funcion: Detecta `git push` exitoso. Extrae SHA y rama, luego lanza `ci-monitor-bg.js` en background (proceso hijo desacoplado). Este daemon pollea GitHub Actions cada 30s durante max 20 minutos y envia

resultado a Telegram.

activity-logger.js

PostToolUse[*]

Funcion: Registra cada herramienta ejecutada (excepto lecturas) en `activity-log.jsonl`. Actualiza la sesion en `sessions/*.json` con conteo de acciones, ultima herramienta, progreso de tasks (sub-pasos). Arranca el reporter PNG automatico si no esta corriendo.

permission-tracker.js

PostToolUse[*]

Funcion: Auto-aprende patrones de permisos exitosos. Si una herramienta se ejecuta sin rechazo, persiste el patron en `settings.local.json` (allow). Esto evita re-preguntar por la misma accion en futuras sesiones. Respeta deny[] explicitos.

post-issue-close.js

PostToolUse[Bash]

Funcion: Detecta `gh issue close`. Via GraphQL, mueve el issue a la columna "Done" en el Project V2 de GitHub. Automatiza la actualizacion del board sin intervencion manual.

notify-telegram.js

Notification

Funcion: Reenvia cualquier notificacion generica de Claude Code a Telegram con formato HTML enriquecido. Incluye contexto del agente activo, rama y tarea actual. Ignora permission_prompts (los maneja permission-approver).

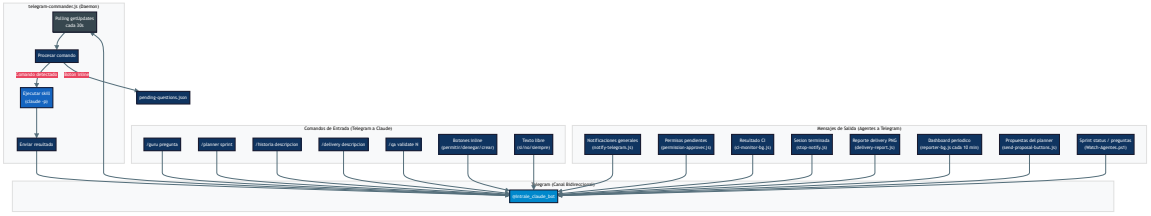
stop-notify.js

Stop

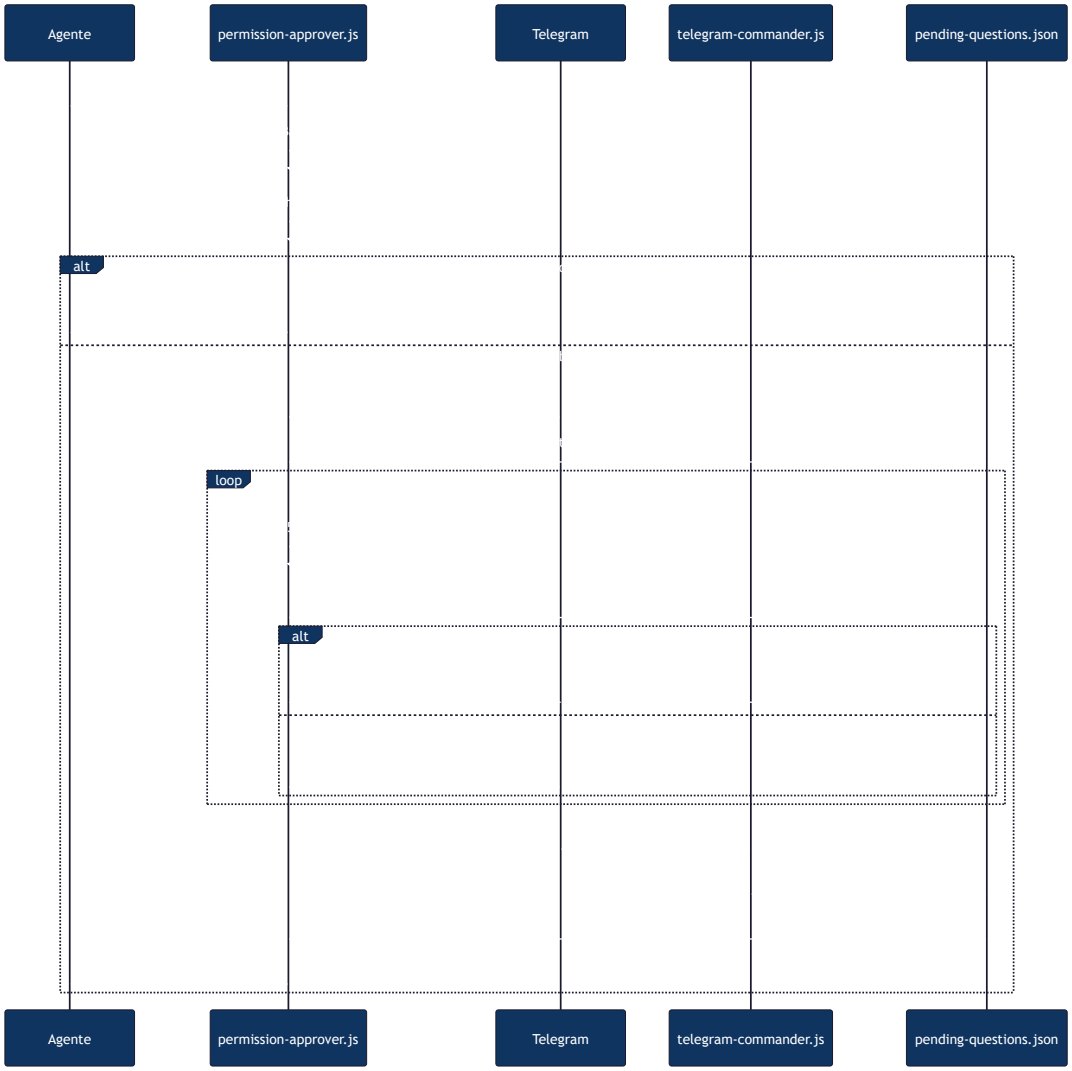
Funcion: Al terminar Claude, marca la sesion como "done" en su JSON, limpia sesiones antiguas (>2h), y envia un resumen final a Telegram (intenta como imagen PNG, fallback a texto HTML).

7. Integracion Telegram

Telegram funciona como canal bidireccional: los agentes envian notificaciones y el usuario puede ejecutar comandos, aprobar permisos y tomar decisiones.



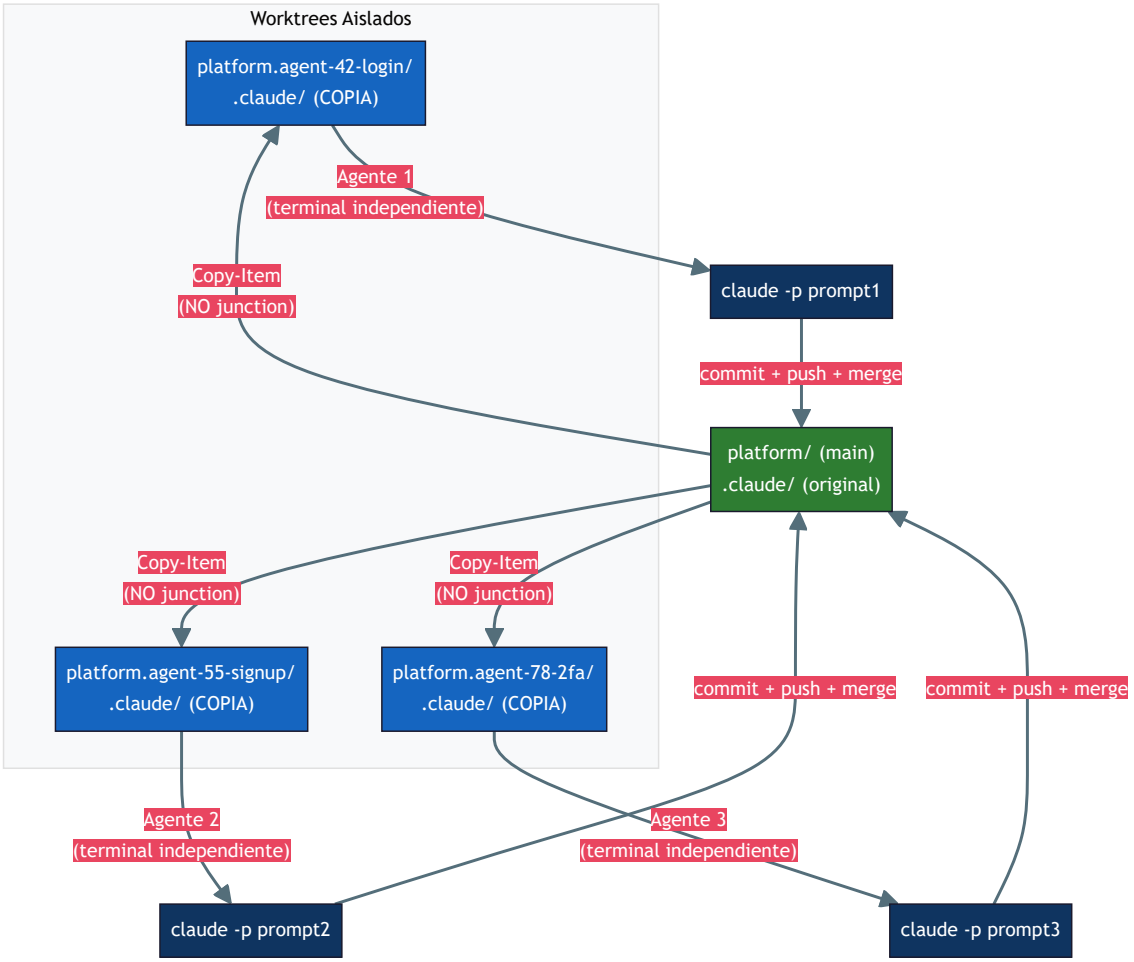
7.1 Flujo de Aprobacion de Permisos



Escalada de urgencia: Si el usuario no responde, los recordatorios suben de intensidad: mensaje inicial (30 min), recordatorio (15 min), agente bloqueado (10 min), ultimo aviso (10 min). Total ~65 minutos antes de caer a prompt local.

8. Orquestacion y Worktrees

8.1 Arquitectura de Worktrees



Leccion historica (2026-02-27): Anteriormente se usaban junctions NTFS para `.claude/`, pero `rm -rf` sobre worktrees borraba el directorio original. La solucion definitiva fue reemplazar junctions con **copias completas** (~500KB, instantaneas).

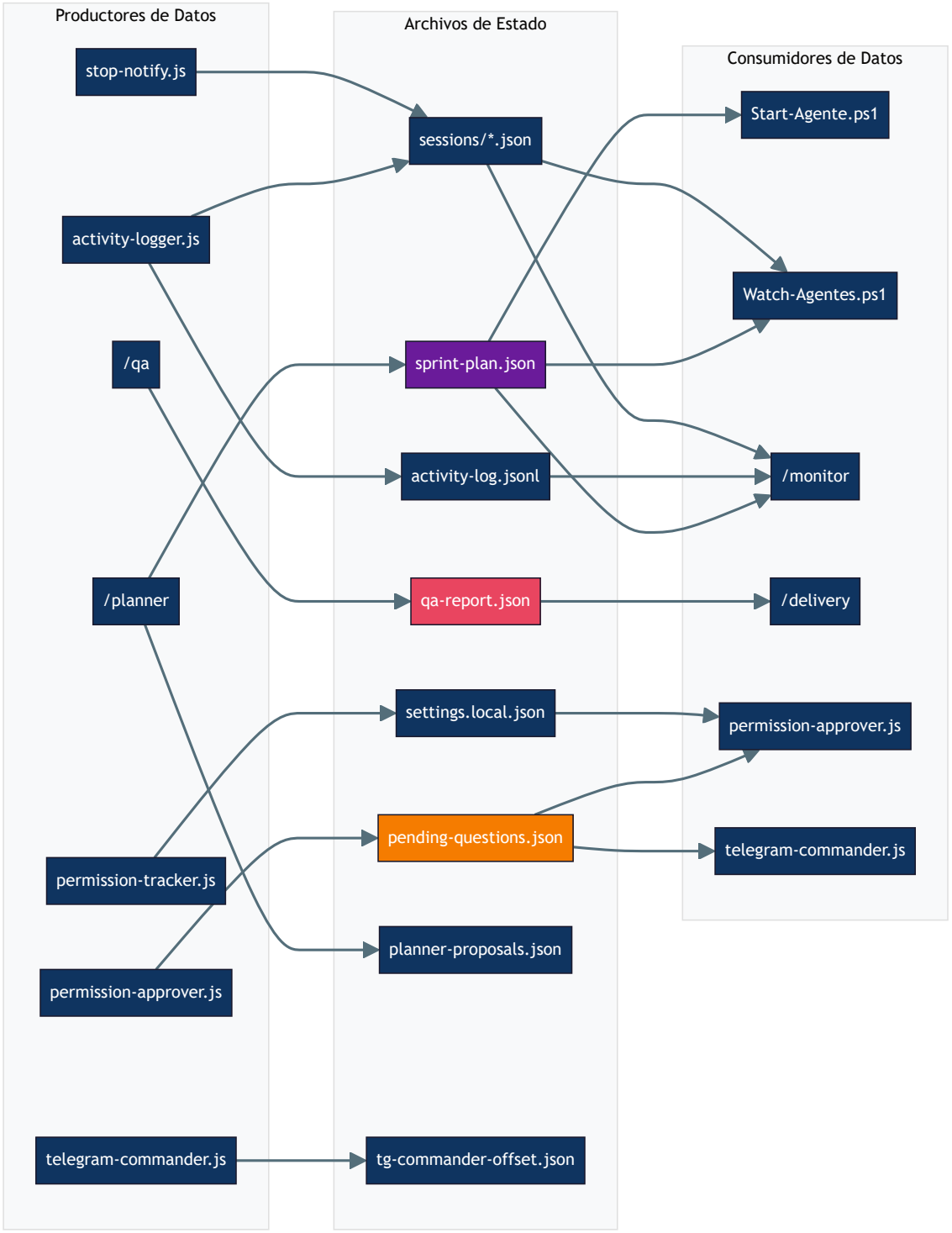
8.2 Scripts de Orquestacion

Script	Tipo	Funcion	Lanzado por
`Start-Agente.ps1`	PowerShell	Lee sprint-plan.json, crea worktrees, abre terminales con agentes, lanza watchers	`/planner sprint`
`Stop-Agente.ps1`	PowerShell	Commit + push + PR + merge por cada agente, cleanup worktrees	`Watch-Agentes.ps1`

`Watch-Agentes.ps1`	PowerShell	Polling cada 30s, detecta terminacion de todos los agentes, dispara Stop + pregunta siguiente sprint	`Start-Agente.ps1 all`
`Guardian-Sprint.ps1`	PowerShell	Keepalive: detecta inactividad total, relanza /planner sprint con cooldown 10 min	`Start-Agente.ps1 all`
`dev-functions.sh`	Bash	CLI local: dev, dev-list, dev-go, dev-done, dev-clean, dev-home	Manual (bashrc)

9. Flujo de Datos Persistentes

El sistema mantiene estado distribuido en archivos JSON que diferentes componentes leen y escriben.



9.1 Tabla de Archivos de Estado

Archivo	Ubicacion	Escrito por	Leido por	Contenido
`sprint-plan.json`	scripts/	/planner	Start/Stop/Watch-Agente, /monitor	Plan del sprint: agentes, issues, prompts, streams

`activity-log.jsonl`	.claude/	activity-logger.js	/monitor, reporter-bg.js	Log de tools ejecutadas (rotado a 500 lineas)
`sessions/*.json`	.claude/sessions/	activity-logger, stop-notify	/monitor, Watch-Agentes	Estado de cada sesion (PID, branch, tasks, progreso)
`pending-questions.json`	.claude/hooks/	permission-approver	telegram-commander	Preguntas de permiso esperando respuesta
`settings.local.json`	.claude/	permission-tracker	permission-approver	Permisos allow/deny auto-aprendidos
`qa-report.json`	qa/evidence/<issue>/	/qa	/delivery	Veredicto QA: APROBADO/RECHAZADO
`planner-proposals.json`	.claude/hooks/	/planner proponer	send-proposal-buttons, telegram-commander	Propuestas de mejora con estado
`sprint-pids.json`	scripts/	Start-Agente	Watch-Agentes	PIDs de agentes lanzados
`telegram-config.json`	.claude/hooks/	Manual	Todos los hooks	bot_token, chat_id, retry intervals
`tg-commander-offset.json`	.claude/hooks/	telegram-commander	telegram-commander	Offset de polling Telegram

10. Modelos y Estrategia de Costos

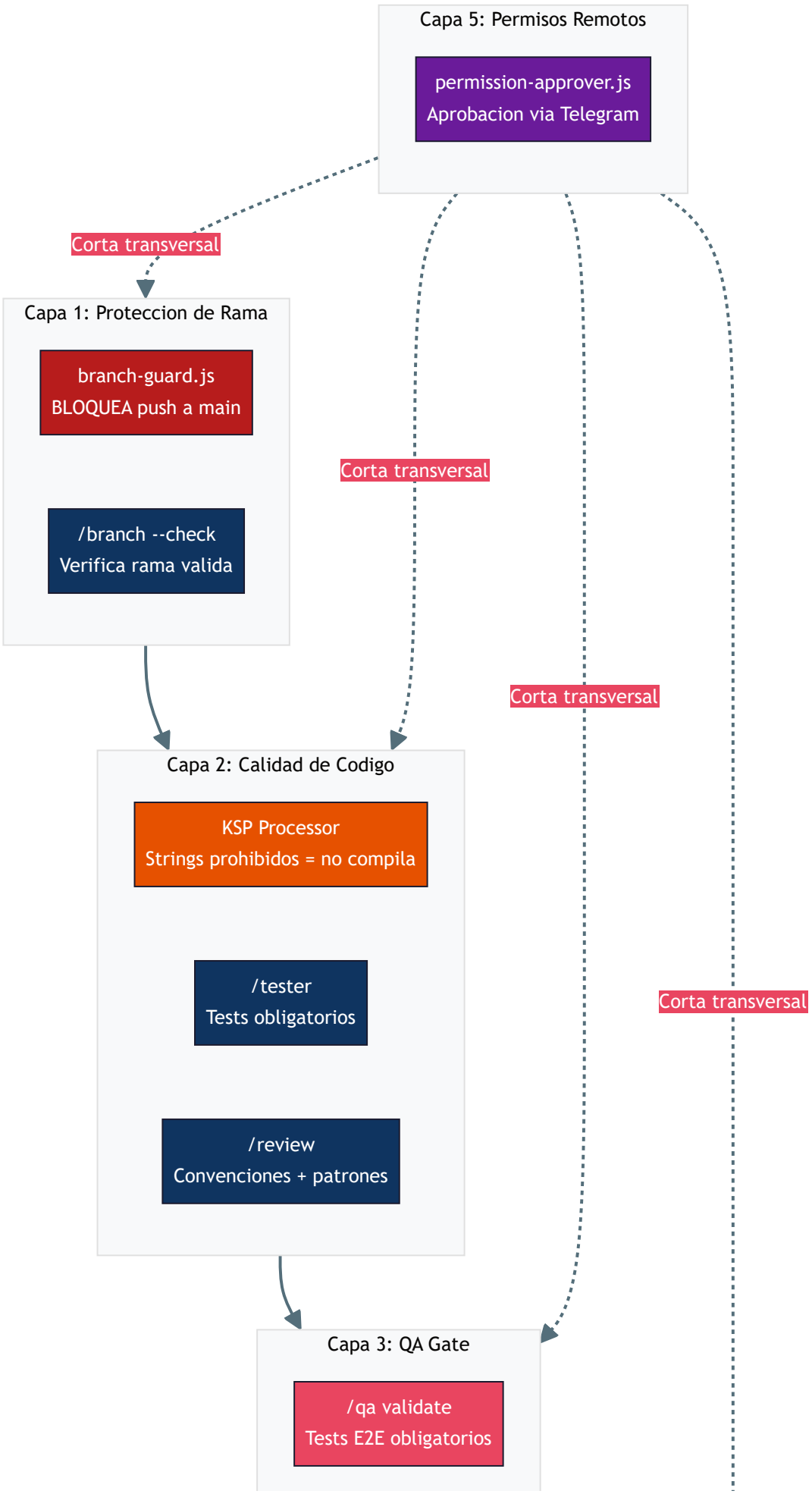
Distribucion de Modelos

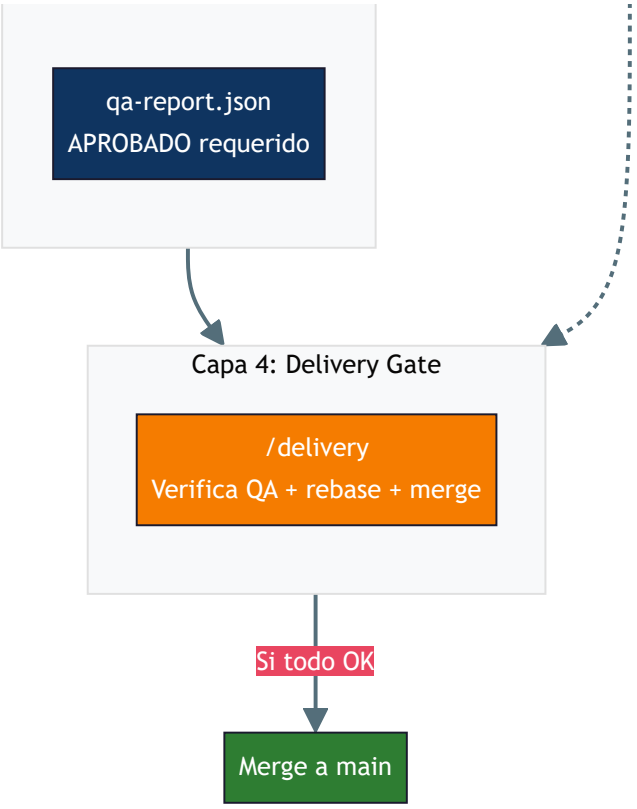
Modelo	Tipo de Tarea	Skills	Cantidad
HAIKU 4.5	Tareas mecanicas, rapidas, bajo costo	/branch, /builder, /tester, /delivery, /monitor, /auth, /cleanup	7
SONNET 4.6	Analisis, sintesis, desarrollo, review	/guru, /planner, /doc, /historia, /refinar, /priorizar, /qa, /review, /po, /ux, /backend-dev, /android-dev, /ios-dev, /web-dev, /desktop-dev	15
OPUS 4.6	Decisiones arquitectonicas complejas	Sesion principal (orquestador)	1

Estrategia: Haiku para tareas repetitivas y mecanicas donde la velocidad importa mas que el razonamiento profundo. Sonnet para tareas que requieren comprension de codigo, analisis de contexto y generacion de contenido complejo. Opus solo para el orquestador principal donde se toman decisiones de alto nivel.

11. Protecciones y Gates

11.1 Capas de Proteccion


Capa 5: Permisos Remotos
permission-approver.js
Aprobacion via Telegram
Corta transversal
Capa 1: Proteccion de Rama
branch-guard.js
BLOQUEA push a main
/branch --check
Verifica rama valida
Corta transversal
Corta transversal
Capa 2: Calidad de Codigo
KSP Processor
Strings prohibidos = no compila
/tester
Tests obligatorios
/review
Convenciones + patrones
Corta transversal
Capa 3: QA Gate
/qa validate
Tests E2E obligatorios



11.2 Resumen de Gates

Gate	Tipo	Bloqueante	Bypass
`branch-guard.js`	Hook automatico	Si (no se puede push a main)	No existe bypass
KSP forbidden-strings	Compilacion	Si (no compila)	No existe bypass
`/qa validate`	Reporte JSON	Si (delivery bloquea)	Solo con confirmacion explicita del usuario
`/delivery` rebase	Conflictos	Parcial (auto-resuelve config, bloquea codigo)	Manual
`permission-approver`	Aprobacion humana	Si (timeout 65 min)	Fallback a prompt local

11.3 Convenciones Criticas

Ramas: Solo se permite `agent/<issue>-<slug>` para agentes IA. Push directo a `main` esta bloqueado por hook automatico. Esto es infalible y no depende de la "memoria" del agente.

Strings: El KSP Processor (`forbidden-strings-processor`) detecta uso de `stringResource()` , `Res.string.*` , `R.string.*` fuera de `ui/util/ResStrings` y **falla la compilacion**. Solo se permite

`resString()` con fallback ASCII-safe.

QA: Antes de cualquier merge, debe existir `qa/evidence/<issue>/qa-report.json` con `"verdict": "APROBADO"`. `/delivery` verifica esto automaticamente y BLOQUEA si no se cumple.

Worktrees: NUNCA usar `rm -rf` sobre directorios de worktree. Siempre usar `git worktree remove --force` seguido de prune. Los `.claude/` son copias completas (no junctions) desde 2026-02-27.

Generado automaticamente — Intrale Platform v1.0 — 2026-03-01

22 skills • 10 hooks • 5 scripts de orquestacion • Ciclo autonomo continuo